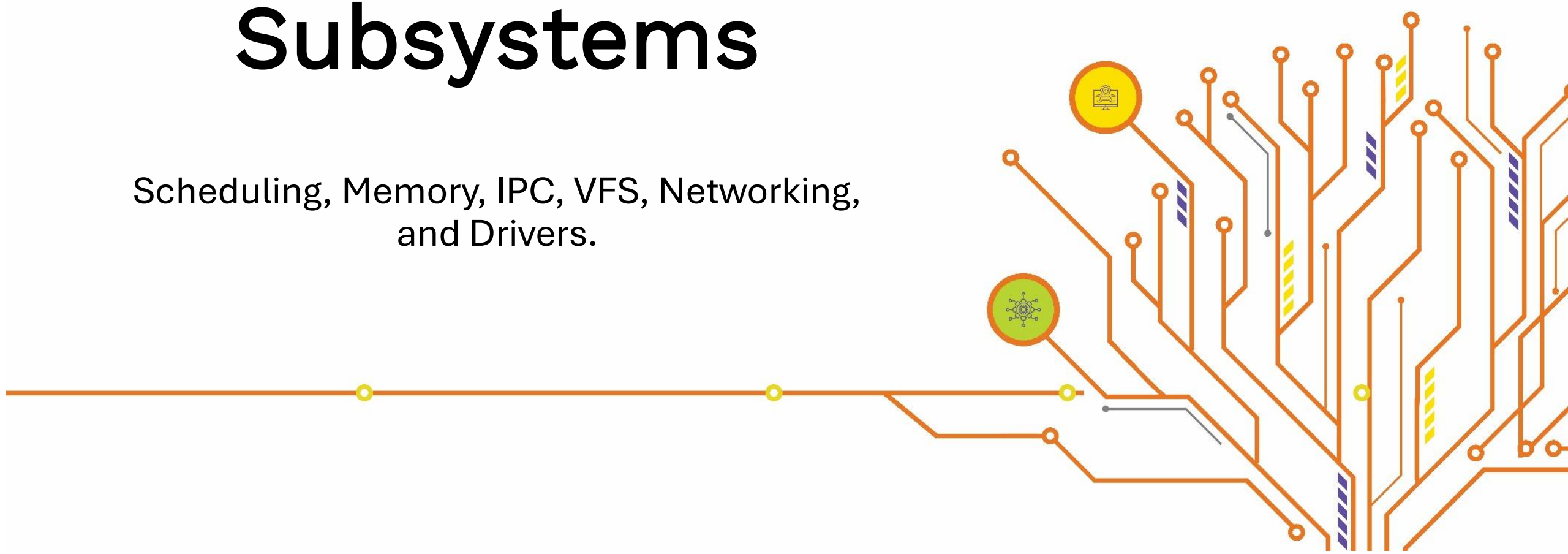
KPIT | APEX LAB

Kernel Subsystems

Scheduling, Memory, IPC, VFS, Networking, and Drivers.



Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

Scheduler

- Ultimate time-manager for your computer's processor
- Decides which process/thread runs when
- Policies:
  - **CFS (Completely Fair Scheduler) – The Default:**
    - This is what Linux uses for most everyday tasks.
    - To give each process a "fair" share of the CPU.
    - It doesn't guarantee equal *time* slices, but equal *runtime*.
    - Processes that have been waiting longer get more CPU time when it's their turn.
  - Real time SCHED_FIFO/SCHED_RR

CFS is not deterministic — if you need guaranteed deadlines, use RT scheduling classes.

DEMO — Scheduling Effect (1/3)

Goal: Show how OS scheduling mediates CPU access and affects timing.

Linux enforces fairness even when tasks misbehave; bare metal assumes your code is perfect.

Fairness vs Guarantees: Linux prioritizes fairness; it does **not** guarantee precise timing at normal priority.

Start two hogs **on the same core**

1. Open terminal and execute

```
$ taskset -c 0 yes > /dev/null &
$ taskset -c 0 yes > /dev/null &
```

This command starts a process that continuously uses CPU time (the yes command) and *forces* it to run exclusively on the first CPU core (core 0), while hiding its output from your screen. It runs this process in the background so you can continue using your terminal.

taskset	set or retrieve the CPU affinity of a running process. Specifying coresa process is allowed to run on
-c 0	CPU core 0 to use
yes	Linux command that continuously outputs the string "y" to standard output. It's intentionally designed to run indefinitely, making it good for testing CPU affinity.
>	It redirects the standard output of a command to a file.
/dev/null	a special file in Linux. Anything written to **/dev/null** is discarded. preventing it from cluttering your terminal
&	tells the shell to run the command in the background.

DEMO — Scheduling Effect (2/3)

2. In another terminal, execute

```
$ top
```

The real time view of a running system (processes) will be shown as below:

```
  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
288021 paragn2  20   0    8368   1920   1920 R  50.4   0.0   0:51.73 yes
288022 paragn2  20   0    8368   1920   1920 R  50.4   0.0   0:51.72 yes
```

Observations

- Look at **%CPU** and **NI** (nice).
- Notice both hogs are around **~50%** each—**fair sharing**."

DEMO — Scheduling Effect (3/3)

Nice: **Higher nice → lower priority** (more positive number = more "polite").
Nice changes the *weight of competition*, not a **hard cap**. A niced task can still show 100% when no one else competes on that core.

3. Reduce the priority of **one** process by making it **more nice** (more positive NI)."

- In terminal 1 execute command

```
$ renice +10 288021
```

288021 is the PID of one process

288021 (process ID) old priority 0, new priority 10

Observations:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
288022	paragn2	20	0	8368	1920	1920	R	90.4	0.0	3:39.11	yes
802	root	20	0	2018816	372980	131328	S	40.9	1.1	17:38.18	pmd
288021	paragn2	30	10	8368	1920	1920	R	9.6	0.0	2:30.44	yes

- Notice **NI** changed to **+10**.
- **Normal yes (NI 0):** ~90%
- **Niced yes (NI +10):** ~10%
- This is **weighted fairness**—not a hard cap. The lower-priority hog runs **less often** even though it's ready to run all the time.

Memory Management

- Memory Management is how the operating system handles the computer's RAM.
- It's responsible for
  - allocating memory to processes,
  - ensuring they don't interfere with each other
  - making efficient use of available resources.

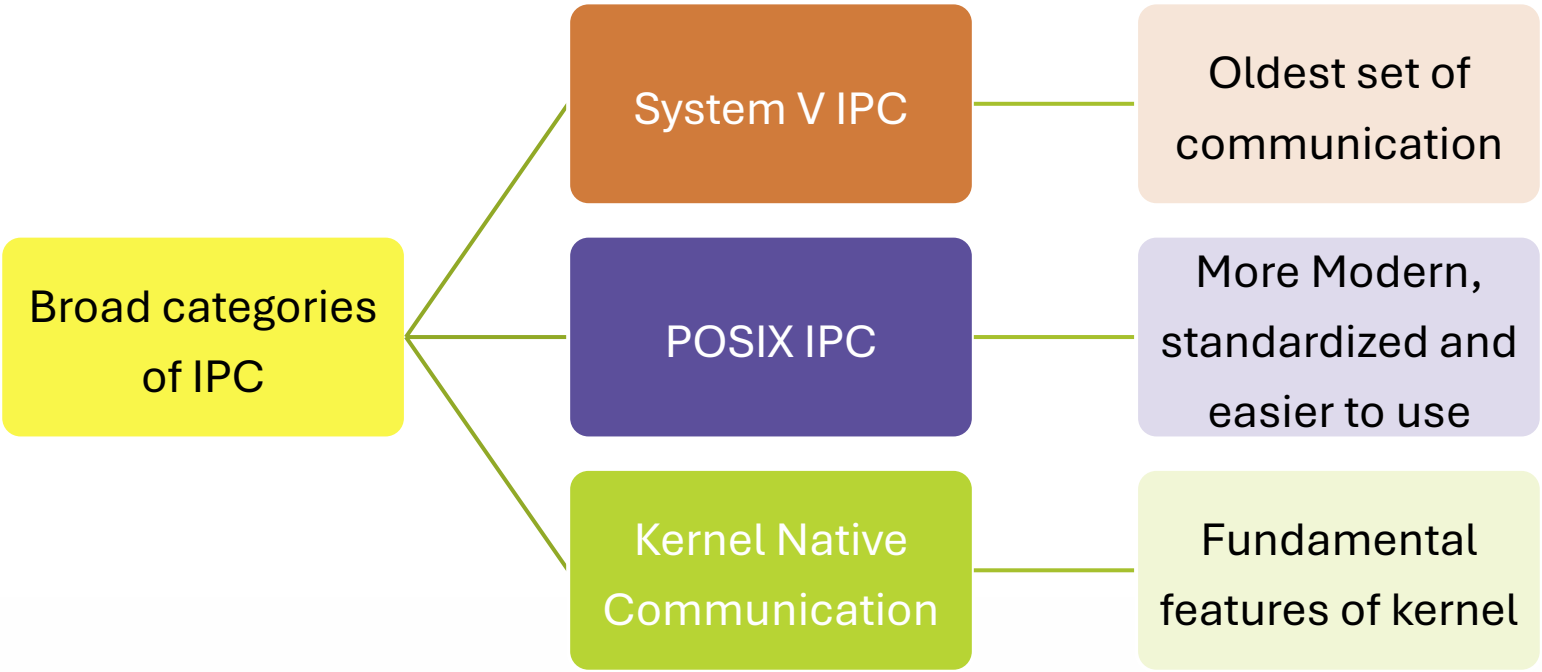
- Every process sees its own private virtual address space.

Memory Management

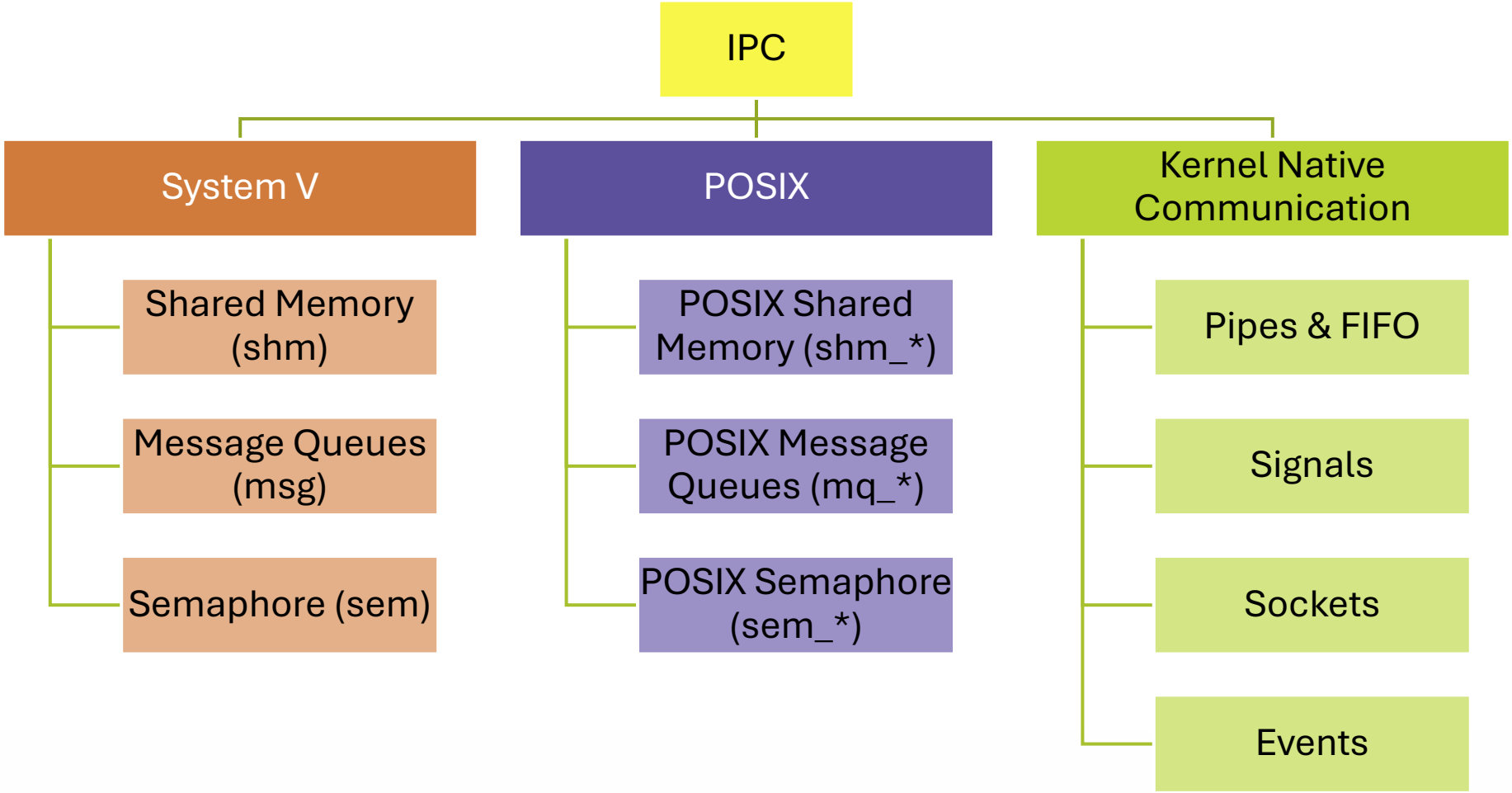
- **The Problem**
  - Physical RAM is limited.
  - Processes often require more memory than is physically available, or it's beneficial to allow processes to use more memory than physically present.
- **The Solution: Virtual Memory**
  - Creates an illusion for each process that it has a large, contiguous block of memory, even if the physical memory is fragmented and smaller.
  - This is done by using disk space as an extension of RAM.
- **How it works:**
  - Processes address memory using *virtual addresses*.
  - The Memory Management Unit (MMU) translates these virtual addresses into *physical addresses* (the actual addresses in RAM).

IPC (InterProcess Communication)

- IPC = Coordination + Communication between processes.
- Kernel manages shared resources.



IPC (InterProcess Communication)



VFS (Virtual Filesystem)

VFS is a major abstraction layer inside Linux kernel that provides unified API for all filesystems

Why VFS exits:

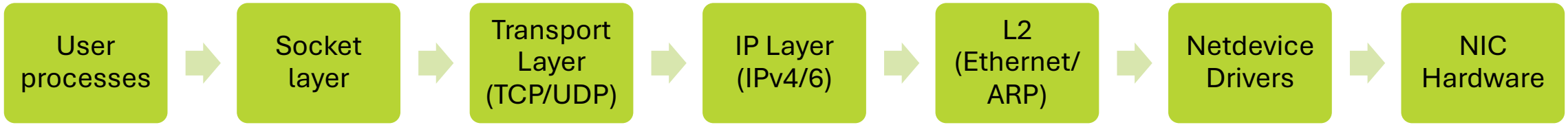
- different filesystems store metadata differently
- But user space expects one consistent API

Example: open() must work same on:

- /home (ext4)
- /sys (sysfs)
- /proc (procfs)
- USB drive (exFAT or FAT32)
- Remote filesystem (NFS)
- Memory filesystem (tmpfs)

Networking

- The Linux networking subsystem (netstack) is one of the largest and most complex parts of Kernel.
- This is the OSI-like stack inside Linux.
- Socket API (TCP/UDP/raw), routing, firewall hooks



Device Drivers

Device Driver is a piece of kernel code that controls hardware or emulates hardware like interfaces, exposing them to user space through standard kernel subsystem:

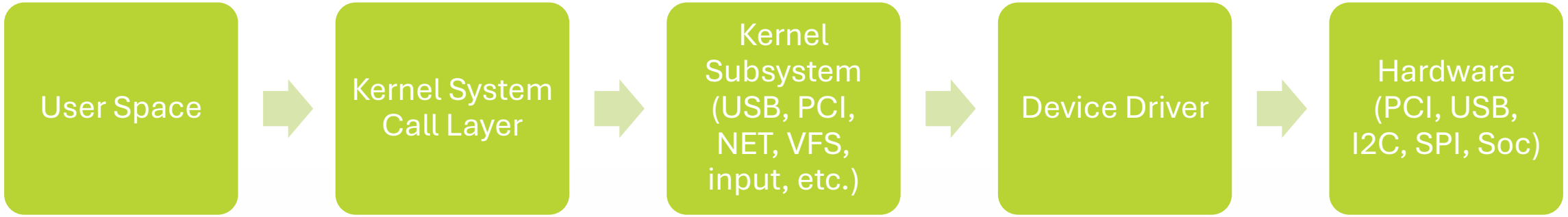
VFS

Networking

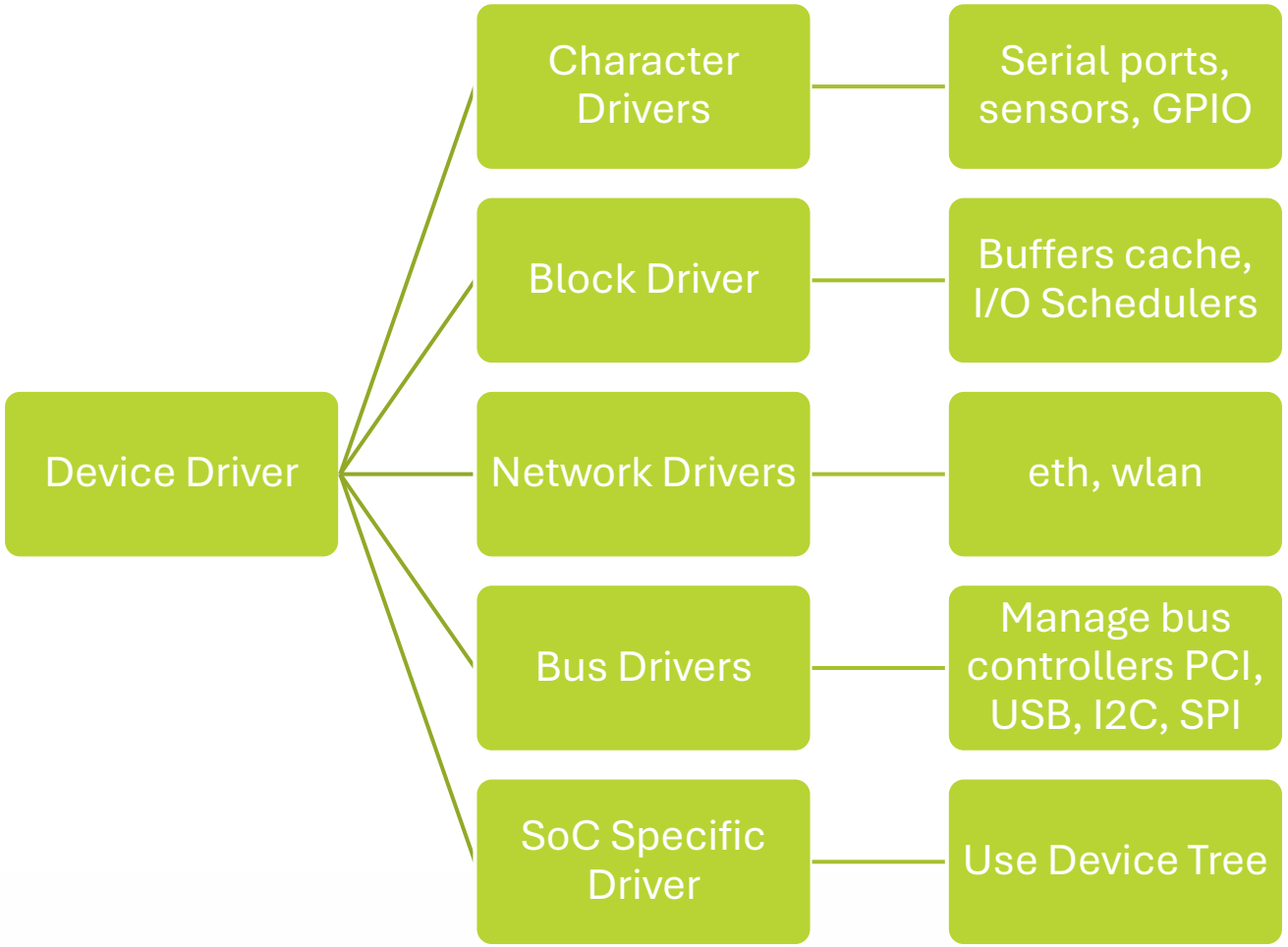
Input subsystem

Character/Block driver framework

- Drivers run in kernel;
- Drivers are NOT the user programs.



Device Drivers



Special Filesystems: /proc, /sys, /dev

These three filesystems provide ways for user-space programs (like your applications) to interact with the kernel.

/proc	/sys (sysfs)	/dev
• The Virtual Process and Kernel Information Hub contains kernel runtime information, not real files	• The Device and Driver Model • sysfs is created directly from kernel objects	• The Device Node Gateway

/proc – The Virtual Process and Kernel Information Hub

/proc is a *virtual* filesystem, t**he files you see in /proc are *not* stored on your hard disk**.

- **Examples:**
  - **/proc/cpuinfo**: Detailed information about your CPU(s) – model, clock speed, features.
  - **/proc/meminfo**: Information about memory usage – total, free, cached, etc.
  - **/proc/<pid>/status**: Status of a specific process (replace **<pid>** with the process ID). Shows state, UID, GID, memory usage, etc.
  - **/proc/<pid>/maps**: Memory map of a process, showing the virtual memory regions it uses (very useful for understanding memory management).
  - **/proc/interrupts**: Shows the number of interrupts handled by each CPU.

- **Use Cases:** System monitoring, debugging processes, gathering kernel statistics.

/proc

```
$ cd /proc
$ ls
```

You'll see a directory for each running process, plus some system-wide info files

```
cat /proc/cpuinfo # Display CPU information
less /proc/meminfo # View memory information (allows scrolling)
```

/sys (sysfs) – The Device and Driver Model

/sys (sysfs) represents the device and driver model of the Linux kernel. It's a structured way to expose information about hardware devices and the drivers that control them.

- **Hierarchical Structure:** Organized in a tree-like structure reflecting the device tree.
- **Attributes:** Devices and drivers expose *attributes* as files within **/sys**. These attributes represent runtime configuration options and the current state of the device.
- **Safe Exposure:** Drivers carefully control which attributes are exposed, preventing accidental or malicious modification of critical settings.
- **Examples:**
  - **/sys/class/gpio**: Information about GPIO pins. enable/disable GPIOs, set their direction (input/output) using attributes within this directory.
  - **/sys/class/net/<iface>**: Information about network interfaces (replace **<iface>** with the interface name, e.g., **eth0**, **wlan0**). Statistics, MAC address, etc.
  - **/sys/bus/i2c/devices/**: Information about I2C devices.
- **Use Cases:** Device configuration, monitoring device status, driver interaction.

/sys

```
$ cd /sys
$ ls
```

see directories like 'class', 'bus', 'devices'

```
cat /sys/class/net/eth0/address # Display the MAC address of eth0
```

/dev – The Device Node Gateway

/dev contains *device nodes*. These are entry points for user-space programs to interact with kernel drivers. Each device node represents a physical or virtual device.

- **Character Devices:** Provide a stream of bytes. Examples: serial ports (**/dev/ttyS0**), keyboard (**/dev/input/event***). Access is typically unbuffered.
- **Block Devices:** Provide access to block-sized data. Examples: hard drives (**/dev/sda**), SSDs (**/dev/nvme0n1**). Access is buffered.
- **Examples:**
  - **/dev/random**, **/dev/urandom**: Random number generators.
  - **/dev/null**: The bit bucket (discards any data written to it).
  - **/dev/sda1**: The first partition on the first SATA drive.
- **Use Cases:** Direct access to hardware, controlling devices, reading data from devices.

/dev

```
cd /dev
ls
```

- interact with devices in **/dev** using specialized commands or programs. Here are a few examples:
- **/dev/null**: Discard output: **echo "Hello" > /dev/null**
- **/dev/random** and **/dev/urandom**: Generate random numbers: **cat /dev/random**
- **/dev/sda1**: Access a disk partition (requires appropriate permissions and tools like **fdisk**, **mount**): **sudo mount /dev/sda1 /mnt**
- **/dev/ttyS0**: Serial port (use tools like **minicom** or **screen**): **sudo minicom -s**

Seeing Inside vs. Entering the Kernel

- **/proc and /sys are how engineers "see inside" the kernel**
  - provide read-only (mostly) access to kernel data and state.
  - *Can observe* what's happening, don't directly control the kernel through these filesystems (except for a few carefully controlled writable attributes in /sys).
- **/dev is how apps "enter" kernel drivers**
  - Device nodes provide the interface for user-space programs to *interact* with kernel drivers.
  - can send commands to the driver, read data from the device, and write data to the device.

Observe → /proc, /sys

Interact → /dev